Backend Development Project Documentation – Node.js

Introduction

Project Title : SpendWise – A Beginner-Friendly Backend Project Using Node.js, MongoDB, and JWT

Team Members

Janathul firthoz.J

Harini s.b

Dhana varshini.S

Janani.D

Project Overview

Purpose

SpendWise is a backend application developed to help beginner developers understand how to create secure REST APIs using Node.js, Express.js, MongoDB, and JWT authentication.The main objective of this project is to demonstrate how user authentication and CRUD operations can be implemented in a simple and structured backend system. This helps learners understand how backend applications operate in real-world environments.

Key Objectives

- Implement secure user authentication using JWT (JSON Web Token)
- Perform CRUD operations to manage transaction records
- Demonstrate database integration using MongoDB and Mongoose
- Implement protected routes using authentication middleware
- Maintain a simple and beginner-friendly backend architecture

Architecture

Backend Component Structure

The backend project follows a modular architecture with separate folders for different functionalities.

server.js

Main entry file that starts the Express server and connects to MongoDB.

Controllers

Controllers handle the business logic of the application, including user authentication and transaction management.

Models

Models define the MongoDB schemas using Mongoose for storing users and transaction data.

Routes

Routes define the API endpoints for authentication and transaction operations.

Middleware

Middleware functions verify JWT tokens and protect private routes from unauthorized access.

Config

Contains configuration files such as database connection and environment variables.

Database Architecture

Users Collection

The Users collection stores user-related information.

Fields include:

- _id
- name
- email
- password (hashed using bcrypt)

  createdAt

Transactions Collection

The Transactions collection stores transaction data created by users.

Fields include:

- _id
- userId (reference to user)
- title or description
- amount
- createdAt

Relationship

- One User can create multiple Transactions
- Each Transaction belongs to one User

Setup Instructions

Prerequisites

- Before running the project, ensure the following tools are installed:
- Node.js (v14 or higher) – JavaScript runtime for backend development
- MongoDB – NoSQL database for storing application data
- npm (Node Package Manager) – Used to install project dependencies
- Visual Studio Code (VS Code) – Recommended editor for development
- Git – Version control system

Installation

Follow the steps below to set up the backend project.

1. Clone the repository

```
git clone <repository-url>
```

2. Navigate to the project folder

```
cd server
```

3. Install dependencies

npm install

4. Install required backend packages

Dependencies used in the project include:

- Express
- Mongoose
- Cors
- Jsonwebtoken
- Bcrypt
- Dotenv

5. Start the development server

npm run dev

6. Access the backend APIs

The backend server runs at:

http://localhost:5000

APIs can be tested using Postman or Thunder Client.

Folder Structure

Server

```
server/
│
├── config/          # Database connection and environment configuration
├── controllers/     # Business logic for authentication and transactions
├── middleware/      # JWT authentication middleware
├── models/          # Mongoose schemas (User, Transaction)
├── routes/          # API routes for auth and transactions
```

```
├── server.js          # Main backend entry point
│
├── package.json       # Project dependencies and scripts
└── .gitignore         # Ignored files for Git
```

Utilities

The backend uses several helper libraries and utilities:

- Axios or Fetch (optional) for API communication
- JWT for authentication and token generation
- bcrypt for password hashing
- dotenv for environment variable management

  cors for handling cross-origin requests

Running the Application

Running the Backend Server

1. Navigate to the backend folder:

cd server

2. Start the development server:

npm run dev

3. The backend server will run at:

http://localhost:5000

You can test API endpoints using Postman or Thunder Client.

API Features

User Authentication

- Register new users
- Login with email and password

- Generate JWT token for authentication

CRUD Operations

Users can perform the following actions on transactions:

- Create a new transaction
- View existing transactions
- Update transaction details
- Delete transactions

Protected Routes

Certain API routes require authentication. These routes are protected using JWT middleware to verify the user's identity.

User Interface

Since SpendWise is a backend-only project, it does not include a frontend interface.

However, APIs can be tested using tools such as:

- Postman
- Thunder Client
- Insomnia

These tools allow developers to send HTTP requests and view responses.

Testing

API Testing

The APIs are tested using Postman and Thunder Client.

Typical tests include:

- User registration
- User login authentication
- Creating transactions

- Fetching transaction data
- Updating transaction records
- Deleting transactions

Error Handling Tests

Testing includes handling errors such as:

- Invalid login credentials
- Unauthorized access without JWT token

  Invalid request parameters

Screenshots or Demo

Demo Link

The demo video of our project:

https://drive.google.com/file/d/1CwN_8AfcnKTLmtCnMGiD9hefZEWEIszq/view?usp=drivesdk

Screenshots

(Add screenshots of API testing using Postman or Thunder Client here)

Known Issues

- Some API responses may take time depending on database connectivity
- Error handling can be improved for more detailed responses

  Rate limiting and advanced security features are not implemented yet

Future Enhancements

- Add a frontend interface using React.js
- Implement role-based authentication
- Add transaction analytics and reporting

- Integrate charts and financial insights
- Improve security and validation mechanisms